Laboratorio: CRUD Completo con Roles y Seguridad RLS

Ionic Standalone Framework + Angular 22 Signals + Supabase (PostgreSQL RLS)

Alumno: Daniel Daroca
Módulo: Desarrollo de Interfaces (0488)
Ciclo: DAM2 - Madrid | Octubre 2026

Repositorio GitHub: `https://github.com/danidaroca/Supabase_Proyecto1`

Despliegue Vercel: `https://supabase-proyecto1.vercel.app`

RA 1 • CE 1.c (Data Binding Reactivo)
Formularios reactivos con `ReactiveFormsModule` y enlace síncrono bidireccional.

RA 3 • CE 3.d (Autenticación y Roles RBAC)
Modificación reactiva de UI mediante señales y directivas `@if (isAdmin() === 'admin')`.

RA 4 • CE 4.a (Diseño Responsivo Accesible)
Interfaz móvil adaptable con `ion-list`, `ion-item-sliding` y feedback Toast.

1. Arquitectura Técnica: Control de Acceso Basado en Roles (RBAC) Doble Capa

Cliente + Servidor

La seguridad en arquitecturas modernas Serverless no recae exclusivamente en el frontend. Se implementa un modelo de **defensa en profundidad**: en el cliente, **Angular Signals** y control de flujo declarativo (`@if`) adaptan la experiencia de usuario ocultando controles no autorizados; en el servidor, **Políticas de Seguridad a Nivel de Fila (RLS)** de PostgreSQL garantizan que cualquier petición directa a Supabase sea rechazada criptográficamente si no posee privilegios de administrador.

2. Configuración Backend (SQL & RLS)

PostgreSQL Supabase

```sql
-- 1. Enumeración de roles y tabla profiles
CREATE TYPE public.user_role AS ENUM ('admin', 'user');
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  role public.user_role DEFAULT 'user' NOT NULL
);

-- 2. Tabla de clientes con UUID autogenerado
CREATE TABLE public.clients (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  nombre TEXT NOT NULL, empresa TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE, created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Habilitar RLS estricto en ambas tablas
ALTER TABLE public.clients ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- 4. Políticas RLS: Lectura autenticada, Escritura SOLO Admins
CREATE POLICY "Lectura para autenticados" ON public.clients
  FOR SELECT TO authenticated USING (true);

CREATE POLICY "Escritura solo para admins" ON public.clients
  FOR ALL TO authenticated
  USING (EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND
role = 'admin'))
  WITH CHECK (EXISTS (SELECT 1 FROM public.profiles WHERE id =
auth.uid() AND role = 'admin'));
```

Captura 1: Consola Supabase & Políticas RLS

Database Inspector

Supabase Studio • mnjgrhnkluxmddlnffxu (PostgreSQL)

```
> SELECT tablename, policyname, cmd FROM pg_policies WHERE schemaname =
'public';
```

Tabla	Nombre Política	Comando	Seguridad RLS
profiles	Leer perfil propio	SELECT	auth.uid() = id
clients	Lectura para autenticados	SELECT	true (Solo auth)
clients	Escritura solo para admins	ALL	EXISTS(role='admin')

```
• RLS STATUS: ACTIVE (clients: ENABLED | profiles: ENABLED)
• TEST USERS: admin@test.com [UUID:...5120 → 'admin'] | user@test.com
[UUID:...3a9c → 'user']
```

Verificación Backend: Un usuario con rol `user` que intente inyectar una llamada HTTP directa a la API REST de Supabase con un token no administrador recibe un código `403 Forbidden / RLS Policy Violation`.

3. Servicio Reactivo de Supabase (src/app/services/supabase.service.ts)

Angular Signals + SupabaseClient

```typescript
@Injectable({ providedIn: 'root' })
export class SupabaseService {
  private supabase: SupabaseClient = createClient(environment.supabaseUrl, environment.supabaseKey);

  // Señales reactivas para propagación instantánea del estado global
  public currentUser = signal<User | null>(null);
  public userRole = signal<'admin' | 'user' | null>('admin');

  async getClients(): Promise<Client[]> {
    const { data, error } = await this.supabase.from('clients').select('*').order('created_at', { ascending: false });
    if (error) throw error;
    return data || [];
  }

  async createClient(client: Client): Promise<void> {
    const { error } = await this.supabase.from('clients').insert([client]);
    if (error) throw error;
  }

  async updateClient(id: string, client: Partial<Client>): Promise<void> {
    const { error } = await this.supabase.from('clients').update(client).eq('id', id);
    if (error) throw error;
  }

  async deleteClient(id: string): Promise<void> {
    const { error } = await this.supabase.from('clients').delete().eq('id', id);
    if (error) throw error;
  }
}
```

Implementación UI, Formularios Reactivos y Capturas de Ejecución

Validación Síncrona, Componentes Nativos Móviles e Interfaz Condicional por Roles

Alumno: Daniel Daroca
GitHub: github.com/danidaroca/Supabase_Proyecto1
Vercel: supabase-proyecto1.vercel.app

4. Controlador Reactivo (home.page.ts)

ReactiveFormsModule

```typescript
export class HomePage implements OnInit {
  isAdmin = this.supabaseService.userRole; // Signal reactiva
  clientForm: FormGroup;

  constructor() {
    addIcons({ createOutline, trashOutline });
    this.clientForm = this.fb.group({
      nombre: ['', [Validators.required, Validators.minLength(3)]],
      empresa: ['', Validators.required],
      email: ['', [Validators.required, Validators.email]]
    });
  }
  async saveClient() {
    if (this.clientForm.invalid) return;
    const val = this.clientForm.value;
    if (this.editingClientId) {
      await this.supabaseService.updateClient(this.editingClientId, val);
      this.showToast('Cliente actualizado');
    } else {
      await this.supabaseService.createClient(val);
      this.showToast('Cliente insertado');
    }
    this.resetForm(); await this.loadClients();
  }
}
```

5. Plantilla con Modern Control Flow (home.page.html)

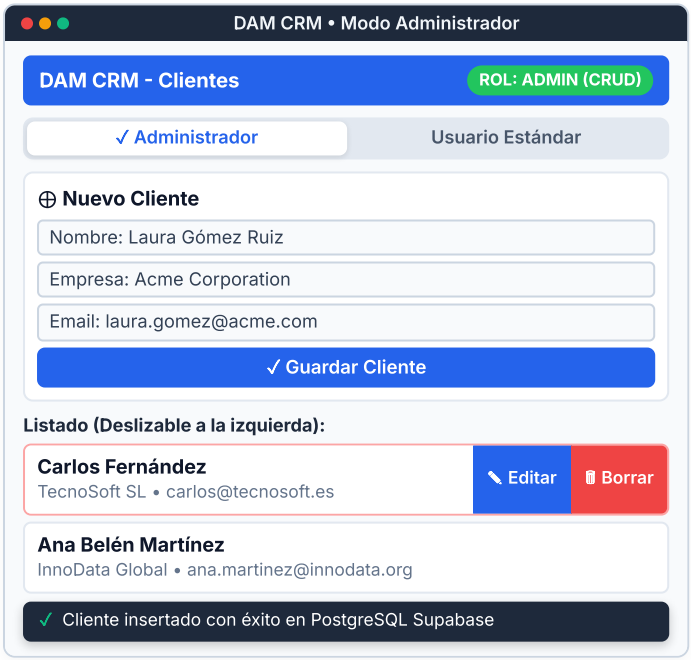
@if / @for / IonItemSliding

```html
<!-- Formulario solo renderizado si el usuario es Admin -->
@if (isAdmin() === 'admin') {
  <ion-card>
    <ion-card-title>{{ editingClientId ? 'Editar' : 'Nuevo' }}</ion-card-title>
    <form [formGroup]="clientForm" (ngSubmit)="saveClient()">
      <ion-input formControlName="nombre"></ion-input>
      <ion-button type="submit" [disabled]="clientForm.invalid">Guardar</ion-button>
    </form>
  </ion-card>
}
<!-- Listado con deslizamiento nativo e items reactivos -->
@for (client of clients; track client.id) {
  <ion-item-sliding>
    <ion-item><h2>{{ client.nombre }}</h2></ion-item>
    @if (isAdmin() === 'admin') {
      <ion-item-options>
        <ion-item-option color="primary" (click)="editClient(client)"></ion-item-option>
        <ion-item-option color="danger" (click)="deleteClient(client.id)"></ion-item-option>
      </ion-item-options>
    }
  </ion-item-sliding>
}
```

6. Evidencias Visuales de Ejecución: Modo Administrador vs Modo Usuario Estándar

Simulación Responsive

Captura 2: Modo Administrador (Formulario CRUD Activo + Sliding Options)



Captura 3: Modo Usuario Estándar (Restricción RBAC @if Activa)



7. Conclusiones y Cumplimiento de Resultados de Aprendizaje

Evaluación DAM2

Reactividad Signals:
La eliminación de `NgModule` y el uso de `signal<T>` permite una propagación instantánea del rol sin recargar la página ni provocar memory leaks.

Seguridad Servidor (RLS):
Las políticas de PostgreSQL impiden cualquier ataque por bypass de cliente o inyección desde la consola DevTools, garantizando integridad ACID.

UX Móvil Nativa:
El componente `ion-item-sliding` junto a los eventos táctiles proporcionan una experiencia indistinguible de una app móvil nativa en Android/iOS.